

The purpose of this assignment was to practice the professional code review process by reviewing another student's Boggle program and my peer reviewing my Boggle program. This assignment gave me experience in looking at code from another perspective of another developer instead of only focusing on whether the code runs correctly. During the review, I considered different areas such as readability, functionality, and PEP 8 guidelines.

My approach was to first read through my partners code and understand their thought process behind how they created their Boggle solver. I then went through the code line by line and provided inline comments with suggestions and also what I liked about their code. After completing my partner's review, I looked through the comments that my partner left for me and decided on what changes I could make while still keeping the originality of my code.

One of the main areas of feedback I received involved making my code less repetitive and easier to understand. My original search method contained separate recursive calls for each of the possible directions surrounding a Boggle tile. My reviewer suggested using loops instead of repeating similar code eight times.

I also received feedback about my variable names. One variable was originally named `lookedAtWords` and it was keeping track of which tiles had already been visited during a search. I changed this variable to `visited`, which more accurately describes its purpose.

When reviewing my partner's code, I identified several strengths and weaknesses. One issue I noticed was that several method names used camelCase instead of snake_case making convention normally recommended by PEP 8. I also suggested adding docstrings because they could provide complete explanations of what different methods do. I looked at input validation as well. For example, I noticed that the program checked the grid for forbidden letters and digits, but there were opportunities to make validation more thorough for unexpected input.

After reading my partner's feedback, I made several changes to my Boggle implementation. I renamed `lookedAtWords` to `visited` so that the variable clearly represents the information it stores. I simplified the handling of special tiles by storing current tiles in a `tile` variable using `len(tile)` to determine how far the word index should move. For example, a normal tile moves index by one character while special tiles move by two. I also replaced the eight individual recursive calls with nested loops that check the eight neighboring positions. The loops skip the current position and recursively search each valid neighboring position. This reduced duplicate code while keeping the same search behavior. Finally, I reset `self.solutions` at the beginning of `getSolution()`. This prevents results from previous calls from remaining in the list and makes repeated calls to the method more reliable.

After making the functional improvements, I ran static analysis on my code. The initial results showed many PEP 8 style violations. These included E11 errors of indentation that was not a multiple of four spaces, E261 and E262 errors involving inline comment formatting, and E501 errors for lines longer than 79 characters. I corrected these issues by fixing the formatting and spacing across multiple lines. I also properly formatted long comments across multiple lines.

After modifying the implementation and formatting, I verified that the changes had not broken the functionality that previously worked. I used the existing unit tests that were created in the previous assignment as regression tests after making the changes. Running multiple tests allowed me to verify that the refactoring preserved the expected behavior of the program.

This assignment helped me understand that code review involves much more than finding code that does not work and professional development workflow. A program can produce the correct output while still having problems with readability, repetition, naming, formatting, and more. Having another person review my code helped me to identify things that I didn't notice.